

CustomerList.jsx

```
deleteCustomer(id) {
  let custs = this.state.customers.filter(c => c.id !== id);
  this.state.customers = custs; // no reconciliation happens
}

// render() returns JSX
render() {
  return (
    <div>
      Name : {this.name} <br />
      {
        this.state.customers.map(cust => <CustomerRow
          deleteEvent={id => this.deleteCustomer(id)}
          key={cust.id}
          customer={cust} />)
      }
    </div>
  );
}
```

export default class CustomerRow extends Component {

```
deleteRow(id) {
  console.log("<CustomerRow /> delete ", id);
  this.props.deleteEvent(id);
}

render() {
  let {id, firstName, lastName} = this.props.customer;
  return (
    <div className='row'>
      {firstName} &nbsp;&nbsp;&nbsp;
      {lastName} &nbsp;&nbsp;&nbsp;
      <button type='button' onClick={() => this.deleteRow(id)}>Delete</button>
    </div>
  );
}
```

Filter the customers:

CustomerList.jsx

```
filterCustomers(txt) {
  let custs = this.state.Original.filter(c => (c.firstName.toUpperCase().indexOf(txt.toUpperCase())
    || (c.lastName.toUpperCase().indexOf(txt.toUpperCase()) >= 0));

  this.setState({
    customers: custs
  });
}
```

<Filter filterEvent={(txt) => this.filterCustomers(txt)} />

```
export default function Filter(props) {
  return (
    <div>
      <input type='text'
        placeholder='search by name'
        onChange={(evt) => props.filterEvent(evt.target.value)}
      />
    </div>
  );
}
```

Change input